

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 8

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SAZID ABDULLAH

STUDENT ID : 23101146

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 10/16/2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following reference string and frame size, implement the FIFO (First In First Out) Page Replacement Algorithm using Python.

- **Reference String**: 1, 2, 3, 4, 2, 3, 1
- **Frame Size: 3**

The program should check whether each page is already present in the available frames. If the page is present, count it as a page hit. Otherwise, count it as a page fault and load the page into the frame. When all frames are full, replace the page that entered the frames first.

Finally, calculate the total number of page hits, page faults, hit ratio, and fault ratio.

2. Source Code:

```
# Page Replacement Algorithm (FIFO)
reference = [1, 2, 3, 4, 2, 3, 1]
frame_size = 3

frame = []
hit = 0
fault = 0

for page in reference:

    if page in frame:
        hit += 1
        print(page, "->", frame, "Hit")

    else:
        fault += 1

        if len(frame) < frame_size:
            frame.append(page)
        else:
            frame.pop(0)
            frame.append(page)
```

```
        print(page, "->", frame, "Fault")

hit_ratio = (hit / len(reference)) * 100
fault_ratio = (fault / len(reference)) * 100

print("Total Hit =", hit)
print("Total Fault =", fault)
print("Hit Ratio =", round(hit_ratio,2),"%")
print("Fault Ratio =", round(fault_ratio, 2),"%")
```

3. Output:



4. Discussion:

This experiment examines the performance of the FIFO (First-In, First-Out) Page Replacement Algorithm using three memory frames and a given page reference sequence. According to this method, when memory becomes full, the page that was loaded earliest is selected for replacement.

At the beginning, pages 1, 2, and 3 occupy the available frames, resulting in three page faults. When page 4 is requested, page 1 is replaced because it entered memory first. The next references, pages 2 and 3, are found in memory,

so they generate page hits. Finally, when page 1 is requested again, another page fault occurs, and page 2 is removed as the oldest remaining page.

After processing the complete reference string, the algorithm records 2 page hits and 5 page faults. The corresponding hit ratio is 28.57%, and the page fault ratio is 71.43%. The outcome illustrates that FIFO bases its replacement decision entirely on the order in which pages entered memory, rather than their usage patterns.

Conclusion:

The FIFO Page Replacement Algorithm was implemented successfully in Python with three memory frames and the reference sequence 1, 2, 3, 4, 2, 3, 1. The simulation produced 5 page faults and 2 page hits, giving a page fault ratio of 71.43% and a hit ratio of 28.57%.

The experiment confirms that FIFO provides a straightforward approach to memory management by removing the page that has been present in memory for the longest period. Its simple logic makes it easy to implement, but the algorithm does not account for how recently or frequently a page has been accessed. As a result, it may replace a page that will be needed again soon. Therefore, FIFO is easy to use, although its effectiveness depends on the page reference sequence and the number of available memory frames.

5. GitHub:

https://github.com/AbirOPS2/CSE402-Operating-System-Lab/tree/main/Lab-08-FIFO-Page-Replacement